CSC111 Project 2 Report: Minks

Caellum Yip Hoi-Lee, Catherine Abdul-Samad, Michael Chen, Joshua Yeung

March 31, 2026

1. Introduction and Research Question

Personal knowledge management (PKM) tools such as Obsidian([**?**]) implement the Zettelkasten methodology([**?**]), allowing users to organize their knowledge as a network of interlinked Markdown notes. A note in the network can link to other notes in the network through internal links called "WikiLinks", forming a graph where the notes are vertices and the links are edges. The benefit of such a system lies in its ability to provide users with insights into the relationship between ideas, and build a stronger understanding of their notes. However, many users (including ourselves), tend to under-link their notes, leading to a lackluster knowledge network. For example, one of the ways one of our group members uses Obsidian is through a "hub-and-spoke" structure, where minimal note vertices known as "hubs" form a directory with WikiLinks to other notes in the same topic (e.g. a hub named "CSC111.md" with WikiLinks to ("Tree.md", "Recursion.md", etc.)). This however, defeats the purpose of Obsidian, failing to capture the lateral connections notes can have in the network which actually make a knowledge graph useful. For instance, a link from "Trees.md" to "Abstract Syntax Trees (CS).md" to "Abstract Syntax Trees (Linguistics).md".

Whilst manually reviewing every note to add missing links is a valuable learning activity, it becomes highly impractical as a PKM system grows to hundreds or thousands of notes.

"Can we use graph-based link prediction algorithms augmented with NLP-based semantic similarity to suggest meaningful missing connections in an Obsidian knowledge graph, and how do the algorithms' judgements compare with a user's own judgement of which notes should be linked?"

2. Datasets

To evaluate our prediction pipeline and avoid circular bias during parameter tuning, we used two distinct datasets:

- **Vault A (Tuning Set):** A personal Obsidian vault belonging to one of the group members, consisting primarily of undergraduate course notes. This dataset was exclusively used in the `MinkPredictor.fit()` method to grid-search the optimal weighting between structural and semantic similarity scores.
- **Vault B (Test Set):** The open-source digital garden of Jacky Zhao (`jzhao.xyz`). This vault contains over 700 notes spanning computer science, philosophy, and other topics. It provides a denser and more diverse graph, allowing us to test our algorithm's out-of-domain generalizability. We utilized the raw `.md` plaintext and internal WikiLinks from this set.

3. Computational Overview

Data Representation

We model an Obsidian vault using a custom `KnowledgeGraph` class, representing the vault as an undirected graph $G = (V, E)$. Each Markdown note is parsed by `load_graph.py` to strip YAML frontmatter and extract plaintext, becoming a `_Note` vertex $v \in V$. WikiLinks between notes become edges $\{u, v\} \in E$. To accomplish this, we implemented a file parser that recursively traverses vault directories, using regular expressions to extract WikiLinks (e.g., `[[Note Name]]`), and clean the raw Markdown for semantic analysis.

Major Computations

Our prediction pipeline, orchestrated by the `MinkPredictor` class, performs the following major computations:

1. **Structural Similarity:** We manually implemented the Jaccard index and the Adamic-Adar index. Adamic-Adar assigns higher weight to shared neighbours with fewer connections, capturing the intuition that a rare mutual connection is stronger evidence for a missing link than a highly connected "hub" note. The output of this index is then normalized to enable standardized comparison. The Jaccard index measures the proportion of shared neighbors relative to the total number of unique neighbors across both notes. It serves as a straightforward, normalized baseline of structural similarity, capturing the intuition that two notes are highly related if a large amount of their overall connections overlap.

2. **Semantic Similarity:** Structural similarity alone fails to detect lateral connections between notes that share context but are **structurally** isolated. To detect these connections, we analyze the semantic content of notes. We utilized a pre-trained sentence-BERT transformer from the `sentence-transformers` library to generate dense, high-dimensional vector embeddings from the parsed note contents. We then calculate Cosine similarity between vectors $S_C = \frac{A \cdot B}{\|A\|\|B\|}$ to accurately evaluate semantic similarity.

3. **Weight Tuning & Holdout Evaluation:** We compute a combined score $S = (w_{struct} \times \text{Structural}) + (w_{sem} \times \text{Semantic})$. During the tuning phase on Vault A, our program automatically performs a grid search across possible weight combinations on our training vault to find the combination that yields the highest recall@k values. We intentionally restricted our search space to a grid of six combinations: $(0.0, 1.0), (0.2, 0.8), (0.4, 0.6), (0.6, 0.4), (0.8, 0.2)$, and $(1.0, 0.0)$. This space prevents an excessive amount of expensive computations that would arise from testing on fine increments. The coarse nature of the grid also prevents accidental overfitting on one specific vault's linking habits, increasing overall generalizability. Once the optimal combination has been discovered, we freeze it and perform repeated edge holdout trials on the test vault. We randomly remove 20% of existing edges, predict the top-k missing links, and calculate average `recall@k` and `precision@k` metrics to evaluate model performance.

Visualization and New Libraries

We used three libraries to report our results interactively:

- **NetworkX:** We convert our custom graph object into a `networkx.Graph` in `visualize.py` to use its `spring_layout()` function, calculates 2D node coordinates based on structural edges.

- **Plotly:** We use `plotly.graph_objects` to render the calculated coordinates into an interactive HTML visualization, visually distinguishing minks' predicted links (red, dashed) from existing WikiLinks (grey, solid).

- **Scikit-Learn (t-SNE):** To compare structural versus semantic clustering, we use `sklearn.manifold.TSNE` to reduce our high-dimensional text embeddings into a 2D space, plotted side-by-side with the NetworkX layout.

4. Instructions for Running the Program

1. **Obtain Datasets:** Download the `vaults.zip` file from [#todo insert link]. Extract the folder so that the `vaults/` directory (containing `vault_a` and `vault_b`) is in the same directory as the `src/` folder.

2. **Install Dependencies:** Run:
`pip install -r src/requirements.txt`. This will install `plotly`, `networkx`, `scikit-learn`, and `sentence-transformer`

3. **Execute:** Run the main pipeline via:
   `cd src`
   `python main.py`.

4. **Expected Output:** The console will print the progress of the weight-fitting grid search on Vault A, followed by the holdout evaluation metrics (Precision/Recall at k) for Vault B. It will output the top predicted missing links and graph statistics (centrality/components). Finally, it will generate two interactive HTML files in the `output/` directory: `vault_b_graph.html` and `layout_comparison.html`. Open these in a web browser to explore the visualizations.

5. Changes to Project Plan

Following feedback on our Phase 1 proposal, we made several critical adjustments to our methodology to improve computational complexity:

- **Preventing Circular Evaluation:** Originally, we planned to evaluate the model on the same vault used for tuning. To prevent this circular bias, we secured a second dataset: Vault A for grid-searching weights, and Vault B as an unseen test set to evaluate out-of-domain generalizability.

- **Manual TF-IDF Implementation:** To ensure our project met the computational complexity requirements, we did not solely rely on the `sentence-transformers` library. We built a TF-IDF embedder from scratch within `similarity.py`, managing term frequencies, inverse document frequencies, and vector normalization natively.

- **Precision Metrics:** We added `precision@k` calculations alongside `recall@k` to better understand the model's accuracy, since suggesting too many irrelevant links (low precision) degrades the utility of a PKM system.

- **Semantic vs. Structural Visual Comparison:** We expanded our visualization pipeline to include t-SNE, directly comparing a layout generated from pure semantic embeddings against the force-directed structural layout to visually determine disparity.

6. Discussion

The results of our computational exploration provide a nuanced answer to our research question: graph-based algorithms augmented with semantic NLP can successfully identify missing connections, but the quantitative definition of a "meaningful" link remains highly subjective.

When evaluated on the tuning set (Vault A), our grid search identified that a combination of structural and semantic similarity (specifically $w_{struct} = 0.4, w_{sem} = 0.6$) outperformed purely structural or purely semantic approaches.

The primary limitation we encountered was the inherent sparsity of the graph and the subjective nature of "missing" links. Our edge holdout evaluation assumes that randomly removed edges are the only valid "correct" predictions. In reality, the algorithm often predicts highly meaningful lateral connections that the user simply hadn't thought to create yet, which the strict evaluation metrics unfairly penalize as "incorrect."

As a next step for further exploration, implementing a "human-in-the-loop" feedback system where users can actively accept or reject suggested links within the interactive GUI would allow the algorithm to dynamically learn user-specific linking preferences over time, overcoming the limitations of static evaluation metrics.